

CSAI2017P — Applied Machine Learning (Theory)

Theory Quiz 3 — Solutions • Set A

Maximum marks 20 • 15 questions on four pages

These are the answers to **Set A**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.

Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

5 × 1 = 5 marks

A1. (c) 1/4

Thirteen of the fifty-two cards are hearts, and $13/52 = 1/4$.

A2. (a) 5

`range(1, 20, 4)` yields 1, 5, 9, 13, 17 — it stops before 20, so 21 is never reached.

A3. (d) n/B

One epoch is one pass through all n examples, consuming them B at a time, so there are n/B batches and therefore n/B updates.

A4. (b) information from the test rows influences training

A step that learns from the data — a scaler, an imputer, a feature selector — must be fitted on the training portion only. Fitted on everything, it has already seen the test rows, and the score that follows is not an honest one.

A5. (c) the running *sum* of squared gradients

Adagrad's $s_t = s_{t-1} + g_t^2$ adds to a running total and never subtracts, so it is a sum. RMSprop's $\gamma s + (1 - \gamma)g^2$ has weights totalling one, which makes it an average.

Part B — Fill in the blanks

4 × 1 = 4 marks

B1. $3w^2$

Bring the exponent down and reduce it by one: $\frac{d}{dw} w^n = n w^{n-1}$, so w^3 differentiates to $3w^2$. Note the derivative is a *function* of w — it is a different number at every point, which is exactly why gradient descent has to re-evaluate it at each step rather than computing it once.

B2. $\sqrt{B}$

A mini-batch gradient is an average of B independent single-example gradients, and averaging B independent quantities divides their standard deviation by $\sqrt{B}$. Measured on the diabetes data: the RMS error was 6.32 at $B = 1$ and 1.06 at $B = 32$, a factor of 5.9 against the predicted $\sqrt{32} = 5.66$. The cost, though, grows like B — which is why $B = 32$ is a good default and $B = 4096$ usually is not.

B3. $\sqrt{t}$

Adagrad's accumulator is a *sum*: $s_t = s_{t-1} + g_t^2$, so it can only ever grow. The step is divided by $\sqrt{s_t}$, and if the gradients stay roughly the same size then s_t grows like t and the step shrinks like $1/\sqrt{t}$ — forever, whether or not you have arrived. Measured: the effective step fell 9.4× over 2800 updates and was still falling. RMSprop's one-line fix is to make s_t an average instead, so it can come back down.

B4. η

Both of Adam's running averages start at zero, so both are biased towards zero early on. Dividing m by $1 - \beta_1^t$ and v by $1 - \beta_2^t$ removes exactly that bias. At $t = 1$ this leaves $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so the step is $\eta g_1 / \sqrt{g_1^2} = \eta \cdot \text{sign}(g_1)$: size exactly η , whatever the gradient was. Without the correction the first step is $\sqrt{1 - \beta_2}/(1 - \beta_1)$ times too big — 3.16× at the usual settings.

Part C — True or false $3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *Data leakage announces itself through a poor score on the held-out data.*

False — and this is what makes leakage dangerous. Leakage means the model saw, at training time, information it will not have at prediction time. That makes the held-out score *better* than the truth, not worse. A disappointing score is an ordinary problem you will go and fix; a suspiciously good one is the problem you will ship. The lecture's demonstration scored 81% on pure noise.

C2. *With the learning rate tuned separately for each, an adaptive optimiser will beat plain gradient descent with momentum on any problem.*

False. Measured on `load_diabetes` with each optimiser at its own tuned learning rate, all six landed within two per cent of each other — and plain momentum won. Adaptive methods shine where coordinates genuinely differ: on the sparse problem the same comparison spanned a factor of eight. What they reliably buy is not a better optimum but a *wider target*: Adagrad worked over 2.24 decades of learning rate against plain SGD's 1.31. That is worth a great deal when you have one afternoon to tune.

C3. *On a ravine-shaped loss surface, plain momentum can converge in fewer iterations than Adam.*

True, and it follows from what each method measures. A ravine is a problem of *curvature* — the loss bends far more steeply across the valley than along it. Adaptive methods do not measure curvature; they measure gradients. Momentum, by accumulating velocity along the consistent direction and cancelling it across the oscillating one, attacks the geometry directly. Measured: 135 iterations for momentum against 262 for Adam.

Part D — Short answer $2 \times 2 = 4$ marks

D1. Both optimisers start from $s_0 = 0$ and take their *first* step with $\eta = 0.5$ and gradient $g_1 = -6$. Adagrad uses $s_t = s_{t-1} + g_t^2$; RMSprop uses $s_t = \gamma s_{t-1} + (1 - \gamma)g_t^2$ with $\gamma = 0.9$. Both then apply $w \leftarrow w - \eta g_t / \sqrt{s_t}$; take ε as negligible.

Both start from $s_0 = 0$, so each accumulator has only g_1 in it.

$$\text{Adagrad: } s_1 = g_1^2 = 36, \quad |\Delta w_1| = \frac{0.5 \times 6}{\sqrt{36}} = \mathbf{0.5}$$

$$\text{RMSprop: } s_1 = (1 - \gamma)g_1^2 = 3.6, \quad |\Delta w_1| = \frac{0.5 \times 6}{\sqrt{3.6}} = \mathbf{1.581139}$$

Adagrad's first step is **exactly** η ; RMSprop's is $\eta / \sqrt{1 - \gamma} = \mathbf{3.162278} \eta$, larger by that factor.

Why neither depends on g_1 . Put the two lines side by side. Adagrad divides ηg_1 by $\sqrt{g_1^2} = |g_1|$, and RMSprop divides it by $|g_1| \sqrt{1 - \gamma}$. Either way the magnitude of g_1 cancels top and bottom, leaving only its *sign* to say which way to move. That is what makes the very first step of an adaptive method a property of the settings rather than of the data — and it is the same cancellation that gives Adam its first step of exactly η . From the second step on the gradients no longer cancel, because s then carries history.

D2. A classifier is trained by minimising cross-entropy, but the figure reported to the client is accuracy. **(a)** Explain why these are two different quantities rather than two names for one. **(b)** State which of them you use to decide the model is finished, and why.

(a) They are different because they answer different questions. The *loss* is what the optimiser minimises, so it must be smooth and differentiable — you need a gradient to descend. The *metric* is what the decision is made on, and it need be neither: accuracy is a step function with zero gradient almost everywhere, which is precisely why it cannot be trained on. They also reward different things. Cross-entropy cares how *confident* a correct prediction was and punishes confident mistakes without bound; accuracy only asks which side of the threshold each prediction landed. A model can lower its cross-entropy without a single prediction changing side, and can improve its accuracy while its cross-entropy gets worse.

(b) The **metric**, computed on held-out data. The slogan is worth keeping: *use the loss to train, use the metric to decide*. And do not compare two models by their losses if those losses are of different kinds — pick one metric and compute it for both on the same held-out rows.

Part E — Reasoning

4 marks

E1. A classmate asserts: “*Adam always beats plain gradient descent with momentum.*” You have one afternoon and a laptop. Design the smallest experiment that would actually settle it.

There is no single right answer here, so what follows is one experiment that would work, and the reasoning that makes it work.

(i) What to run it on. The claim says *always*, so a single dataset cannot establish it and a single counter-example can destroy it. Run at least two problems of deliberately different character — one dense and well conditioned, one sparse or badly conditioned — because that is exactly the axis on which the two methods are known to differ.

(ii) What to measure, and when. Final held-out loss (or the metric you actually care about) at a fixed budget — a fixed number of epochs, or a fixed wall-clock time, fixed in advance. Repeat over a few seeds and report the spread: two runs differing by less than the seed-to-seed variation have not differed at all.

(iii) The thing that must be held equal. The learning rate must be tuned separately for each optimiser. This is the whole item. Adam and momentum have different best learning rates — they are not even on the same scale — so running both at one shared η measures only which one happens to suit that η . Sweep η for each, take each method’s best, and compare those. Everything else (initialisation, batch size, budget, seeds, data splits) is held fixed.

(iv) What would decide it. Accept the claim only if Adam wins on *every* problem by more than the seed-to-seed spread — that is what *always* demands. Reject it the moment momentum wins on even one, by more than that spread. Anything in between is the honest answer that the claim is too strong: it depends on the problem.

CSAI2017P — Applied Machine Learning (Theory)

Theory Quiz 3 — Solutions • Set B

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set B**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.

Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

5 × 1 = 5 marks

A1. (d) 1/4

Five of the twenty items are marked, and $5/20 = 1/4$.

A2. (b) 3

`xs[1:4]` takes positions 1, 2 and 3, stopping before 4: three elements.

A3. (a) 125

$n/B = 4000/32 = 125$ batches in one pass, and one update per batch.

A4. (c) it is optimistic — test information reached the model

The scaler was fitted on all 500 rows, so it already knew the test rows' means and spreads. The 0.99 is therefore flattering, and the honest figure is unknown until the scaler is refitted inside the split.

A5. (d) Adagrad

A sum can only grow; an average can fall again. That single difference is the whole distinction between Adagrad and RMSprop.

Part B — Fill in the blanks

4 × 1 = 4 marks

B1. $3w^2$

Bring the exponent down and reduce it by one: $\frac{d}{dw} w^n = n w^{n-1}$, so w^3 differentiates to $3w^2$. Note the derivative is a *function* of w — it is a different number at every point, which is exactly why gradient descent has to re-evaluate it at each step rather than computing it once.

B2. $\sqrt{B}$

A mini-batch gradient is an average of B independent single-example gradients, and averaging B independent quantities divides their standard deviation by $\sqrt{B}$. Measured on the diabetes data: the RMS error was 6.32 at $B = 1$ and 1.06 at $B = 32$, a factor of 5.9 against the predicted $\sqrt{32} = 5.66$. The cost, though, grows like B — which is why $B = 32$ is a good default and $B = 4096$ usually is not.

B3. $\sqrt{t}$

Adagrad's accumulator is a *sum*: $s_t = s_{t-1} + g_t^2$, so it can only ever grow. The step is divided by $\sqrt{s_t}$, and if the gradients stay roughly the same size then s_t grows like t and the step shrinks like $1/\sqrt{t}$ — forever, whether or not you have arrived. Measured: the effective step fell 9.4× over 2800 updates and was still falling. RMSprop's one-line fix is to make s_t an average instead, so it can come back down.

B4. η

Both of Adam's running averages start at zero, so both are biased towards zero early on. Dividing m by $1 - \beta_1^t$ and v by $1 - \beta_2^t$ removes exactly that bias. At $t = 1$ this leaves $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so the step is $\eta g_1 / \sqrt{g_1^2} = \eta \cdot \text{sign}(g_1)$: size exactly η , whatever the gradient was. Without the correction the first step is $\sqrt{1 - \beta_2}/(1 - \beta_1)$ times too big — 3.16× at the usual settings.

Part C — True or false $3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *A results table in which the held-out score is worse than expected is the signature of data leakage.*

False — and this is what makes leakage dangerous. Leakage means the model saw, at training time, information it will not have at prediction time. That makes the held-out score *better* than the truth, not worse. A disappointing score is an ordinary problem you will go and fix; a suspiciously good one is the problem you will ship. The lecture's demonstration scored 81% on pure noise.

C2. *A table comparing six optimisers, each at its own tuned learning rate, must show an adaptive method at the top.*

False. Measured on `load_diabetes` with each optimiser at its own tuned learning rate, all six landed within two per cent of each other — and plain momentum won. Adaptive methods shine where coordinates genuinely differ: on the sparse problem the same comparison spanned a factor of eight. What they reliably buy is not a better optimum but a *wider target*: Adagrad worked over 2.24 decades of learning rate against plain SGD's 1.31. That is worth a great deal when you have one afternoon to tune.

C3. *A table reading 135 iterations for momentum against 262 for Adam on a ravine is possible.*

True, and it follows from what each method measures. A ravine is a problem of *curvature* — the loss bends far more steeply across the valley than along it. Adaptive methods do not measure curvature; they measure gradients. Momentum, by accumulating velocity along the consistent direction and cancelling it across the oscillating one, attacks the geometry directly. Measured: 135 iterations for momentum against 262 for Adam.

Part D — Short answer $2 \times 2 = 4$ marks

D1. The table below is to be completed for the *first* step of each optimiser from $s_0 = 0$, with $\eta = 0.2$, $\gamma = 0.75$ and $g_1 = +4$. Adagrad accumulates $s_t = s_{t-1} + g_t^2$; RMSprop accumulates $s_t = \gamma s_{t-1} + (1 - \gamma)g_t^2$; both then apply $w \leftarrow w - \eta g_t / \sqrt{s_t}$, with ε negligible.

Optimiser	s_1	$ \Delta w_1 $
Adagrad		
RMSprop		

Both start from $s_0 = 0$, so each accumulator has only g_1 in it.

$$\text{Adagrad: } s_1 = g_1^2 = 16, \quad |\Delta w_1| = \frac{0.2 \times 4}{\sqrt{16}} = \mathbf{0.2}$$

$$\text{RMSprop: } s_1 = (1 - \gamma)g_1^2 = 4, \quad |\Delta w_1| = \frac{0.2 \times 4}{\sqrt{4}} = \mathbf{0.4}$$

Adagrad's first step is **exactly** η ; RMSprop's is $\eta / \sqrt{1 - \gamma} = \mathbf{2\eta}$, larger by that factor.

Why neither depends on g_1 . Put the two lines side by side. Adagrad divides ηg_1 by $\sqrt{g_1^2} = |g_1|$, and RMSprop divides it by $|g_1| \sqrt{1 - \gamma}$. Either way the magnitude of g_1 cancels top and bottom, leaving only its *sign* to say which way to move. That is what makes the very first step of an adaptive method a property of the settings rather than of the data — and it is the same cancellation that gives Adam its first step of exactly η . From the second step on the gradients no longer cancel, because s then carries history.

D2. A results table carries two columns for the same classifier: a cross-entropy column, which the training loop minimised, and an accuracy column, which is what the client was shown. **(a)** Explain why these are two different quantities rather than two names for one. **(b)** State which of them you use to decide the model is finished, and why.

(a) They are different because they answer different questions. The *loss* is what the optimiser minimises, so it must be smooth and differentiable — you need a gradient to descend. The *metric* is what the decision is made on, and it need be neither: accuracy is a step function with zero gradient almost everywhere, which is precisely why it cannot be trained on. They also reward different things. Cross-entropy cares how *confident* a correct prediction was and punishes confident mistakes without bound; accuracy only asks which side of the threshold each prediction landed. A model can lower its cross-entropy without a single prediction changing side, and can improve its accuracy while its cross-entropy gets worse.

(b) The **metric**, computed on held-out data. The slogan is worth keeping: *use the loss to train, use the metric to decide*. And do not compare two models by their losses if those losses are of different kinds — pick one metric and compute it for both on the same held-out rows.

Part E — Reasoning

4 marks

E1. A results table circulating in the lab is being used to argue that “*Adam always beats plain gradient descent with momentum*”. You have one afternoon and a laptop. Design the smallest experiment that would actually settle it.

There is no single right answer here, so what follows is one experiment that would work, and the reasoning that makes it work.

(i) What to run it on. The claim says *always*, so a single dataset cannot establish it and a single counter-example can destroy it. Run at least two problems of deliberately different character — one dense and well conditioned, one sparse or badly conditioned — because that is exactly the axis on which the two methods are known to differ.

(ii) What to measure, and when. Final held-out loss (or the metric you actually care about) at a fixed budget — a fixed number of epochs, or a fixed wall-clock time, fixed in advance. Repeat over a few seeds and report the spread: two runs differing by less than the seed-to-seed variation have not differed at all.

(iii) The thing that must be held equal. The learning rate must be tuned separately for each optimiser. This is the whole item. Adam and momentum have different best learning rates — they are not even on the same scale — so running both at one shared η measures only which one happens to suit that η . Sweep η for each, take each method’s best, and compare those. Everything else (initialisation, batch size, budget, seeds, data splits) is held fixed.

(iv) What would decide it. Accept the claim only if Adam wins on *every* problem by more than the seed-to-seed spread — that is what *always* demands. Reject it the moment momentum wins on even one, by more than that spread. Anything in between is the honest answer that the claim is too strong: it depends on the problem.

CSAI2017P — Applied Machine Learning (Theory)

Theory Quiz 3 — Solutions • Set C

Maximum marks 20 • 15 questions on four pages

These are the answers to **Set C**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.

Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

5 × 1 = 5 marks

A1. (a) 1/4

Two of the eight equal sectors are shaded, and $2/8 = 1/4$.

A2. (c) 3

A **set** keeps each distinct character once, and *banana* contains only *b*, *a* and *n*.

A3. (b) 125

One epoch is one pass through all 4000 rows, taken 32 at a time, so there are $4000/32 = 125$ batches and one update per batch. The other two bars would stand at 4000 (for $B = 1$) and 1 (for full batch).

A4. (d) a fitted step that saw the whole dataset before the split

A score far above what the problem admits is the signature of leakage, not of success. The lecture's demonstration reached 81% on data with no signal in it at all.

A5. (a) Adagrad

$\sqrt{s_t}$ rising forever is Adagrad's defining flaw: the denominator only grows, so the step only shrinks. RMSprop's denominator can come back down.

Part B — Fill in the blanks

4 × 1 = 4 marks

B1. $3w^2$

Bring the exponent down and reduce it by one: $\frac{d}{dw} w^n = n w^{n-1}$, so w^3 differentiates to $3w^2$. Note the derivative is a *function* of w — it is a different number at every point, which is exactly why gradient descent has to re-evaluate it at each step rather than computing it once.

B2. $\sqrt{B}$

A mini-batch gradient is an average of B independent single-example gradients, and averaging B independent quantities divides their standard deviation by $\sqrt{B}$. Measured on the diabetes data: the RMS error was 6.32 at $B = 1$ and 1.06 at $B = 32$, a factor of 5.9 against the predicted $\sqrt{32} = 5.66$. The cost, though, grows like B — which is why $B = 32$ is a good default and $B = 4096$ usually is not.

B3. $\sqrt{t}$

Adagrad's accumulator is a *sum*: $s_t = s_{t-1} + g_t^2$, so it can only ever grow. The step is divided by $\sqrt{s_t}$, and if the gradients stay roughly the same size then s_t grows like t and the step shrinks like $1/\sqrt{t}$ — forever, whether or not you have arrived. Measured: the effective step fell 9.4× over 2800 updates and was still falling. RMSprop's one-line fix is to make s_t an average instead, so it can come back down.

B4. η

Both of Adam's running averages start at zero, so both are biased towards zero early on. Dividing m by $1 - \beta_1^t$ and v by $1 - \beta_2^t$ removes exactly that bias. At $t = 1$ this leaves $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so the step is $\eta g_1 / \sqrt{g_1^2} = \eta \cdot \text{sign}(g_1)$: size exactly η , whatever the gradient was. Without the correction the first step is $\sqrt{1 - \beta_2}/(1 - \beta_1)$ times too big — 3.16× at the usual settings.

Part C — True or false $3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *On a plot of held-out score against epoch, data leakage shows up as a curve sitting below where it should.*

False — and this is what makes leakage dangerous. Leakage means the model saw, at training time, information it will not have at prediction time. That makes the held-out score *better* than the truth, not worse. A disappointing score is an ordinary problem you will go and fix; a suspiciously good one is the problem you will ship. The lecture's demonstration scored 81% on pure noise.

C2. *If six optimisers are each plotted at their own best learning rate, the adaptive ones must end up below the non-adaptive ones.*

False. Measured on `load_diabetes` with each optimiser at its own tuned learning rate, all six landed within two per cent of each other — and plain momentum won. Adaptive methods shine where coordinates genuinely differ: on the sparse problem the same comparison spanned a factor of eight. What they reliably buy is not a better optimum but a *wider target*: Adagrad worked over 2.24 decades of learning rate against plain SGD's 1.31. That is worth a great deal when you have one afternoon to tune.

C3. *A trajectory plot in which momentum reaches the minimum of a narrow valley in fewer steps than Adam is possible.*

True, and it follows from what each method measures. A ravine is a problem of *curvature* — the loss bends far more steeply across the valley than along it. Adaptive methods do not measure curvature; they measure gradients. Momentum, by accumulating velocity along the consistent direction and cancelling it across the oscillating one, attacks the geometry directly. Measured: 135 iterations for momentum against 262 for Adam.

Part D — Short answer $2 \times 2 = 4$ marks

D1. Two optimisers are to be drawn taking their *first* step from the same point, with $s_0 = 0$, $\eta = 0.4$ and a gradient of $g_1 = -10$ at that point. Adagrad accumulates $s_t = s_{t-1} + g_t^2$; RMSprop accumulates $s_t = \gamma s_{t-1} + (1 - \gamma)g_t^2$ with $\gamma = 0.84$. Both then apply $w \leftarrow w - \eta g_t / \sqrt{s_t}$; take ε as negligible.

Both start from $s_0 = 0$, so each accumulator has only g_1 in it.

$$\text{Adagrad: } s_1 = g_1^2 = 100, \quad |\Delta w_1| = \frac{0.4 \times 10}{\sqrt{100}} = \mathbf{0.4}$$

$$\text{RMSprop: } s_1 = (1 - \gamma)g_1^2 = 16, \quad |\Delta w_1| = \frac{0.4 \times 10}{\sqrt{16}} = \mathbf{1}$$

Adagrad's first step is **exactly** η ; RMSprop's is $\eta / \sqrt{1 - \gamma} = \mathbf{2.5} \eta$, larger by that factor.

Why neither depends on g_1 . Put the two lines side by side. Adagrad divides ηg_1 by $\sqrt{g_1^2} = |g_1|$, and RMSprop divides it by $|g_1| \sqrt{1 - \gamma}$. Either way the magnitude of g_1 cancels top and bottom, leaving only its *sign* to say which way to move. That is what makes the very first step of an adaptive method a property of the settings rather than of the data — and it is the same cancellation that gives Adam its first step of exactly η . From the second step on the gradients no longer cancel, because s then carries history.

D2. Two curves are plotted against the epoch: the cross-entropy the training loop was minimising, and the accuracy that was reported to the client. They do not have the same shape. **(a)** Explain why these are two different quantities rather than two names for one. **(b)** State which of them you use to decide the model is finished, and why.

(a) They are different because they answer different questions. The *loss* is what the optimiser minimises, so it must be smooth and differentiable — you need a gradient to descend. The *metric* is what the decision is made on, and it need be neither: accuracy is a step function with zero gradient almost everywhere, which is precisely why it cannot be trained on. They also reward different things. Cross-entropy cares how *confident* a correct prediction was and punishes confident mistakes without bound; accuracy only asks which side of the threshold each prediction landed. A model can lower its cross-entropy without a single prediction changing side, and can improve its accuracy while its cross-entropy gets worse.

(b) The **metric**, computed on held-out data. The slogan is worth keeping: *use the loss to train, use the metric to decide*. And do not compare two models by their losses if those losses are of different kinds — pick one metric and compute it for both on the same held-out rows.

Part E — Reasoning

4 marks

E1. A plot is being passed around as proof that “*Adam always beats plain gradient descent with momentum*”. You have one afternoon and a laptop. Design the smallest experiment that would actually settle it.

There is no single right answer here, so what follows is one experiment that would work, and the reasoning that makes it work.

(i) What to run it on. The claim says *always*, so a single dataset cannot establish it and a single counter-example can destroy it. Run at least two problems of deliberately different character — one dense and well conditioned, one sparse or badly conditioned — because that is exactly the axis on which the two methods are known to differ.

(ii) What to measure, and when. Final held-out loss (or the metric you actually care about) at a fixed budget — a fixed number of epochs, or a fixed wall-clock time, fixed in advance. Repeat over a few seeds and report the spread: two runs differing by less than the seed-to-seed variation have not differed at all.

(iii) The thing that must be held equal. The learning rate must be tuned separately for each optimiser. This is the whole item. Adam and momentum have different best learning rates — they are not even on the same scale — so running both at one shared η measures only which one happens to suit that η . Sweep η for each, take each method’s best, and compare those. Everything else (initialisation, batch size, budget, seeds, data splits) is held fixed.

(iv) What would decide it. Accept the claim only if Adam wins on *every* problem by more than the seed-to-seed spread — that is what *always* demands. Reject it the moment momentum wins on even one, by more than that spread. Anything in between is the honest answer that the claim is too strong: it depends on the problem.

CSAI2017P — Applied Machine Learning (Theory)

Theory Quiz 3 — Solutions • Set D

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set D**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.

Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

 $5 \times 1 = 5$ marks**A1. (b) $1/4$**

Ten of the forty students cycle, and $10/40 = 1/4$.

A2. (d) 2

A dictionary cannot hold the same key twice — the second 'tea' overwrites the first, leaving two entries.

A3. (c) 125 times

$4000/32 = 125$ batches in one pass, and the weights move once per batch.

A4. (a) optimistic — the test rows helped fit the scaler

Scaling is not innocent: it is a step *fitted* to data, so fitting it on everything lets the test rows influence training. The figure is optimistic.

A5. (b) Adadelta

Adadelta replaces the learning rate in the numerator with $\text{RMS}[\Delta w]$, which fixes the units and removes η altogether. It found its own step of 0.0124 where a human had grid-searched to 0.005.

Part B — Fill in the blanks

 $4 \times 1 = 4$ marks**B1. $3w^2$**

Bring the exponent down and reduce it by one: $\frac{d}{dw} w^n = n w^{n-1}$, so w^3 differentiates to $3w^2$. Note the derivative is a *function* of w — it is a different number at every point, which is exactly why gradient descent has to re-evaluate it at each step rather than computing it once.

B2. $\sqrt{B}$

A mini-batch gradient is an average of B independent single-example gradients, and averaging B independent quantities divides their standard deviation by $\sqrt{B}$. Measured on the diabetes data: the RMS error was 6.32 at $B = 1$ and 1.06 at $B = 32$, a factor of 5.9 against the predicted $\sqrt{32} = 5.66$. The cost, though, grows like B — which is why $B = 32$ is a good default and $B = 4096$ usually is not.

B3. $\sqrt{t}$

Adagrad's accumulator is a *sum*: $s_t = s_{t-1} + g_t^2$, so it can only ever grow. The step is divided by $\sqrt{s_t}$, and if the gradients stay roughly the same size then s_t grows like t and the step shrinks like $1/\sqrt{t}$ — forever, whether or not you have arrived. Measured: the effective step fell $9.4\times$ over 2800 updates and was still falling. RMSprop's one-line fix is to make s_t an average instead, so it can come back down.

B4. η

Both of Adam's running averages start at zero, so both are biased towards zero early on. Dividing m by $1 - \beta_1^t$ and v by $1 - \beta_2^t$ removes exactly that bias. At $t = 1$ this leaves $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so the step is $\eta g_1 / \sqrt{g_1^2} = \eta \cdot \text{sign}(g_1)$: size exactly η , whatever the gradient was. Without the correction the first step is $\sqrt{1 - \beta_2}/(1 - \beta_1)$ times too big — $3.16\times$ at the usual settings.

Part C — True or false $3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *A team whose held-out score came out disappointingly low should suspect data leakage.*

False — and this is what makes leakage dangerous. Leakage means the model saw, at training time, information it will not have at prediction time. That makes the held-out score *better* than the truth, not worse. A disappointing score is an ordinary problem you will go and fix; a suspiciously good one is the problem you will ship. The lecture's demonstration scored 81% on pure noise.

C2. *A manager who has tuned the learning rate for every optimiser on the shortlist can be confident the adaptive ones will win.*

False. Measured on `load_diabetes` with each optimiser at its own tuned learning rate, all six landed within two per cent of each other — and plain momentum won. Adaptive methods shine where coordinates genuinely differ: on the sparse problem the same comparison spanned a factor of eight. What they reliably buy is not a better optimum but a *wider target*: Adagrad worked over 2.24 decades of learning rate against plain SGD's 1.31. That is worth a great deal when you have one afternoon to tune.

C3. *An engineer working on a long narrow valley may find plain momentum reaches the bottom in fewer iterations than Adam.*

True, and it follows from what each method measures. A ravine is a problem of *curvature* — the loss bends far more steeply across the valley than along it. Adaptive methods do not measure curvature; they measure gradients. Momentum, by accumulating velocity along the consistent direction and cancelling it across the oscillating one, attacks the geometry directly. Measured: 135 iterations for momentum against 262 for Adam.

Part D — Short answer $2 \times 2 = 4$ marks

D1. An engineer runs the same model twice from the same starting weights, once with Adagrad and once with RMSprop, and compares only the *first* step. Both start from $s_0 = 0$ with $\eta = 0.25$, and the gradient at the starting point is $g_1 = +2$. Adagrad accumulates $s_t = s_{t-1} + g_t^2$; RMSprop accumulates $s_t = \gamma s_{t-1} + (1 - \gamma)g_t^2$ with $\gamma = 0.9375$. Both apply $w \leftarrow w - \eta g_t / \sqrt{s_t}$; take ε as negligible.

Both start from $s_0 = 0$, so each accumulator has only g_1 in it.

$$\text{Adagrad: } s_1 = g_1^2 = 4, \quad |\Delta w_1| = \frac{0.25 \times 2}{\sqrt{4}} = \mathbf{0.25}$$

$$\text{RMSprop: } s_1 = (1 - \gamma)g_1^2 = 0.25, \quad |\Delta w_1| = \frac{0.25 \times 2}{\sqrt{0.25}} = \mathbf{1}$$

Adagrad's first step is **exactly** η ; RMSprop's is $\eta / \sqrt{1 - \gamma} = \mathbf{4\eta}$, larger by that factor.

Why neither depends on g_1 . Put the two lines side by side. Adagrad divides ηg_1 by $\sqrt{g_1^2} = |g_1|$, and RMSprop divides it by $|g_1| \sqrt{1 - \gamma}$. Either way the magnitude of g_1 cancels top and bottom, leaving only its *sign* to say which way to move. That is what makes the very first step of an adaptive method a property of the settings rather than of the data — and it is the same cancellation that gives Adam its first step of exactly η . From the second step on the gradients no longer cancel, because s then carries history.

D2. A client has been shown an accuracy figure for a spam filter. The training loop that produced the filter never used accuracy at all — it minimised cross-entropy. **(a)** Explain why these are two different quantities rather than two names for one. **(b)** State which of them you use to decide the model is finished, and why.

(a) They are different because they answer different questions. The *loss* is what the optimiser minimises, so it must be smooth and differentiable — you need a gradient to descend. The *metric* is what the decision is made on, and it need be neither: accuracy is a step function with zero gradient almost everywhere, which is precisely why it cannot be trained on. They also reward different things. Cross-entropy cares how *confident* a correct prediction was and punishes confident mistakes without bound; accuracy only asks which side of the threshold each prediction landed. A model can lower its cross-entropy without a single prediction changing side, and can improve its accuracy while its cross-entropy gets worse.

(b) The **metric**, computed on held-out data. The slogan is worth keeping: *use the loss to train, use the metric to decide*. And do not compare two models by their losses if those losses are of different kinds — pick one metric and compute it for both on the same held-out rows.

Part E — Reasoning

4 marks

E1. A colleague insists, in a design meeting, that “*Adam always beats plain gradient descent with momentum*” and wants it written into the team’s defaults. You have one afternoon and a laptop. Design the smallest experiment that would actually settle it.

There is no single right answer here, so what follows is one experiment that would work, and the reasoning that makes it work.

(i) What to run it on. The claim says *always*, so a single dataset cannot establish it and a single counter-example can destroy it. Run at least two problems of deliberately different character — one dense and well conditioned, one sparse or badly conditioned — because that is exactly the axis on which the two methods are known to differ.

(ii) What to measure, and when. Final held-out loss (or the metric you actually care about) at a fixed budget — a fixed number of epochs, or a fixed wall-clock time, fixed in advance. Repeat over a few seeds and report the spread: two runs differing by less than the seed-to-seed variation have not differed at all.

(iii) The thing that must be held equal. The learning rate must be tuned separately for each optimiser. This is the whole item. Adam and momentum have different best learning rates — they are not even on the same scale — so running both at one shared η measures only which one happens to suit that η . Sweep η for each, take each method’s best, and compare those. Everything else (initialisation, batch size, budget, seeds, data splits) is held fixed.

(iv) What would decide it. Accept the claim only if Adam wins on *every* problem by more than the seed-to-seed spread — that is what *always* demands. Reject it the moment momentum wins on even one, by more than that spread. Anything in between is the honest answer that the claim is too strong: it depends on the problem.

CSAI2017P — Applied Machine Learning (Theory)

Theory Quiz 3 — Solutions • Set E

Maximum marks 20 • 15 questions on four pages

These are the answers to **Set E**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.

Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

5 × 1 = 5 marks

A1. (c) 0.25

Thirteen of the fifty-two cards are hearts, so the expression is $13/52 = 0.25$.

A2. (b) 3

`range(12)` runs 0 to 11, and the multiples of four among them are 0, 4 and 8 — remember to count zero.

A3. (a) 125 times

The loader yields $\lceil 4000/32 \rceil = 125$ batches, and the body runs once per batch.

A4. (d) optimistic — the test rows helped fit the scaler

`fit_transform` on the whole of `X` fits the scaler to every row, including the ones `train_test_split` is about to hold back.

A5. (d) RMSprop

`s = s + g**2` accumulates without decay (Adagrad); `s = gamma*s + (1-gamma)*g**2` is an exponentially weighted average (RMSprop). The second line of both updates is identical.

Part B — Fill in the blanks

4 × 1 = 4 marks

B1. $3w^2$

Bring the exponent down and reduce it by one: $\frac{d}{dw} w^n = n w^{n-1}$, so w^3 differentiates to $3w^2$. Note the derivative is a *function* of w — it is a different number at every point, which is exactly why gradient descent has to re-evaluate it at each step rather than computing it once.

B2. $\sqrt{B}$

A mini-batch gradient is an average of B independent single-example gradients, and averaging B independent quantities divides their standard deviation by $\sqrt{B}$. Measured on the diabetes data: the RMS error was 6.32 at $B = 1$ and 1.06 at $B = 32$, a factor of 5.9 against the predicted $\sqrt{32} = 5.66$. The cost, though, grows like B — which is why $B = 32$ is a good default and $B = 4096$ usually is not.

B3. $\sqrt{t}$

Adagrad's accumulator is a *sum*: $s_t = s_{t-1} + g_t^2$, so it can only ever grow. The step is divided by $\sqrt{s_t}$, and if the gradients stay roughly the same size then s_t grows like t and the step shrinks like $1/\sqrt{t}$ — forever, whether or not you have arrived. Measured: the effective step fell 9.4× over 2800 updates and was still falling. RMSprop's one-line fix is to make s_t an average instead, so it can come back down.

B4. η

Both of Adam's running averages start at zero, so both are biased towards zero early on. Dividing m by $1 - \beta_1^t$ and v by $1 - \beta_2^t$ removes exactly that bias. At $t = 1$ this leaves $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so the step is $\eta g_1 / \sqrt{g_1^2} = \eta \cdot \text{sign}(g_1)$: size exactly η , whatever the gradient was. Without the correction the first step is $\sqrt{1 - \beta_2}/(1 - \beta_1)$ times too big — 3.16× at the usual settings.

Part C — True or false $3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *`model.score(X_test, y_test)` coming back lower than expected is the signature of data leakage.*

False — and this is what makes leakage dangerous. Leakage means the model saw, at training time, information it will not have at prediction time. That makes the held-out score *better* than the truth, not worse. A disappointing score is an ordinary problem you will go and fix; a suspiciously good one is the problem you will ship. The lecture's demonstration scored 81% on pure noise.

C2. *With lr tuned for each, Adam must reach a lower final loss than `SGD(momentum=0.9)`.*

False. Measured on `load_diabetes` with each optimiser at its own tuned learning rate, all six landed within two per cent of each other — and plain momentum won. Adaptive methods shine where coordinates genuinely differ: on the sparse problem the same comparison spanned a factor of eight. What they reliably buy is not a better optimum but a *wider target*: Adagrad worked over 2.24 decades of learning rate against plain SGD's 1.31. That is worth a great deal when you have one afternoon to tune.

C3. *On a badly conditioned quadratic, `SGD(momentum=0.9)` can need fewer iterations than Adam.*

True, and it follows from what each method measures. A ravine is a problem of *curvature* — the loss bends far more steeply across the valley than along it. Adaptive methods do not measure curvature; they measure gradients. Momentum, by accumulating velocity along the consistent direction and cancelling it across the oscillating one, attacks the geometry directly. Measured: 135 iterations for momentum against 262 for Adam.

Part D — Short answer $2 \times 2 = 4$ marks

D1. The two loops below are each run for exactly one step from $\mathbf{s} = 0.0$, with $\mathbf{eta} = 0.1$ and a first gradient of $\mathbf{g} = -5.0$; $\mathbf{gamma} = 0.96$ and $\mathbf{eps}$ is negligible.

<code># Adagrad</code>	<code># RMSprop</code>
<code>s = s + g**2</code>	<code>s = gamma*s + (1 - gamma)*g**2</code>
<code>w = w - eta*g/np.sqrt(s)</code>	<code>w = w - eta*g/np.sqrt(s)</code>

Both start from $s_0 = 0$, so each accumulator has only g_1 in it.

$$\text{Adagrad: } s_1 = g_1^2 = 25, \quad |\Delta w_1| = \frac{0.1 \times 5}{\sqrt{25}} = \mathbf{0.1}$$

$$\text{RMSprop: } s_1 = (1 - \gamma)g_1^2 = 1, \quad |\Delta w_1| = \frac{0.1 \times 5}{\sqrt{1}} = \mathbf{0.5}$$

Adagrad's first step is **exactly** η ; RMSprop's is $\eta/\sqrt{1-\gamma} = 5\eta$, larger by that factor.

Why neither depends on g_1 . Put the two lines side by side. Adagrad divides ηg_1 by $\sqrt{g_1^2} = |g_1|$, and RMSprop divides it by $|g_1|\sqrt{1-\gamma}$. Either way the magnitude of g_1 cancels top and bottom, leaving only its *sign* to say which way to move. That is what makes the very first step of an adaptive method a property of the settings rather than of the data — and it is the same cancellation that gives Adam its first step of exactly η . From the second step on the gradients no longer cancel, because s then carries history.

D2. The training loop minimises `nn.CrossEntropyLoss()`, but the number in the report comes from `accuracy_score(...)`. **(a)** Explain why these are two different quantities rather than two names for one. **(b)** State which of them you use to decide the model is finished, and why.

(a) They are different because they answer different questions. The *loss* is what the optimiser minimises, so it must be smooth and differentiable — you need a gradient to descend. The *metric* is what the decision is made on, and it need be neither: accuracy is a step function with zero gradient almost everywhere, which is precisely why it cannot be trained on. They also reward different things. Cross-entropy cares how *confident* a correct prediction was and punishes confident mistakes without bound; accuracy only asks which side of the threshold each prediction landed. A model can lower its cross-entropy without a single prediction changing side, and can improve its accuracy while its cross-entropy gets worse.

(b) The **metric**, computed on held-out data. The slogan is worth keeping: *use the loss to train, use the metric to decide*. And do not compare two models by their losses if those losses are of different kinds — pick one metric and compute it for both on the same held-out rows.

Part E — Reasoning

4 marks

E1. A pull request changes the project default from `SGD(momentum=0.9)` to `Adam`, with the justification “*Adam always beats plain gradient descent with momentum*”. You have one afternoon and a laptop. Design the smallest experiment that would actually settle it.

There is no single right answer here, so what follows is one experiment that would work, and the reasoning that makes it work.

(i) What to run it on. The claim says *always*, so a single dataset cannot establish it and a single counter-example can destroy it. Run at least two problems of deliberately different character — one dense and well conditioned, one sparse or badly conditioned — because that is exactly the axis on which the two methods are known to differ.

(ii) What to measure, and when. Final held-out loss (or the metric you actually care about) at a fixed budget — a fixed number of epochs, or a fixed wall-clock time, fixed in advance. Repeat over a few seeds and report the spread: two runs differing by less than the seed-to-seed variation have not differed at all.

(iii) The thing that must be held equal. The learning rate must be tuned separately for each optimiser. This is the whole item. Adam and momentum have different best learning rates — they are not even on the same scale — so running both at one shared η measures only which one happens to suit that η . Sweep η for each, take each method’s best, and compare those. Everything else (initialisation, batch size, budget, seeds, data splits) is held fixed.

(iv) What would decide it. Accept the claim only if Adam wins on *every* problem by more than the seed-to-seed spread — that is what *always* demands. Reject it the moment momentum wins on even one, by more than that spread. Anything in between is the honest answer that the claim is too strong: it depends on the problem.

CSAI2017P — Applied Machine Learning (Theory)

Theory Quiz 3 — Solutions • Set F

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set F**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.

Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

5 × 1 = 5 marks

A1. (d) one in four

One suit in four is hearts, so the chance is one in four.

A2. (c) 5

Counting up in fours from one: 1, 5, 9, 13, 17 — five numbers below twenty.

A3. (b) one hundred and twenty-five

Four thousand divided by thirty-two is one hundred and twenty-five batches, and the weights move once per batch.

A4. (a) better than it should be

Anything fitted to the data before the split has already seen the part you meant to hold back, so the number that comes out flatters the model.

A5. (a) Adadelta

Adadelta asks for no step size because it supplies one itself, from the sizes of the steps it has already taken. That also makes the units come out right, which is the argument it was invented for.

Part B — Fill in the blanks

4 × 1 = 4 marks

B1. $3w^2$

Bring the exponent down and reduce it by one: $\frac{d}{dw} w^n = n w^{n-1}$, so w^3 differentiates to $3w^2$. Note the derivative is a *function* of w — it is a different number at every point, which is exactly why gradient descent has to re-evaluate it at each step rather than computing it once.

B2. $\sqrt{B}$

A mini-batch gradient is an average of B independent single-example gradients, and averaging B independent quantities divides their standard deviation by $\sqrt{B}$. Measured on the diabetes data: the RMS error was 6.32 at $B = 1$ and 1.06 at $B = 32$, a factor of 5.9 against the predicted $\sqrt{32} = 5.66$. The cost, though, grows like B — which is why $B = 32$ is a good default and $B = 4096$ usually is not.

B3. $\sqrt{t}$

Adagrad's accumulator is a *sum*: $s_t = s_{t-1} + g_t^2$, so it can only ever grow. The step is divided by $\sqrt{s_t}$, and if the gradients stay roughly the same size then s_t grows like t and the step shrinks like $1/\sqrt{t}$ — forever, whether or not you have arrived. Measured: the effective step fell 9.4× over 2800 updates and was still falling. RMSprop's one-line fix is to make s_t an average instead, so it can come back down.

B4. η

Both of Adam's running averages start at zero, so both are biased towards zero early on. Dividing m by $1 - \beta_1^t$ and v by $1 - \beta_2^t$ removes exactly that bias. At $t = 1$ this leaves $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so the step is $\eta g_1 / \sqrt{g_1^2} = \eta \cdot \text{sign}(g_1)$: size exactly η , whatever the gradient was. Without the correction the first step is $\sqrt{1 - \beta_2}/(1 - \beta_1)$ times too big — 3.16× at the usual settings.

Part C — True or false $3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

- C1.** *When something that should not have been seen is seen during training, the tell-tale sign is a disappointing score on the data held back.*

False — and this is what makes leakage dangerous. Leakage means the model saw, at training time, information it will not have at prediction time. That makes the held-out score *better* than the truth, not worse. A disappointing score is an ordinary problem you will go and fix; a suspiciously good one is the problem you will ship. The lecture's demonstration scored 81% on pure noise.

- C2.** *Once the step size has been chosen carefully for each method in turn, the ones that adapt per coordinate are bound to come out ahead.*

False. Measured on `load_diabetes` with each optimiser at its own tuned learning rate, all six landed within two per cent of each other — and plain momentum won. Adaptive methods shine where coordinates genuinely differ: on the sparse problem the same comparison spanned a factor of eight. What they reliably buy is not a better optimum but a *wider target*: Adagrad worked over 2.24 decades of learning rate against plain SGD's 1.31. That is worth a great deal when you have one afternoon to tune.

- C3.** *On a long narrow valley, carrying momentum can get you to the bottom in fewer steps than a method that adapts each coordinate separately.*

True, and it follows from what each method measures. A ravine is a problem of *curvature* — the loss bends far more steeply across the valley than along it. Adaptive methods do not measure curvature; they measure gradients. Momentum, by accumulating velocity along the consistent direction and cancelling it across the oscillating one, attacks the geometry directly. Measured: 135 iterations for momentum against 262 for Adam.

Part D — Short answer $2 \times 2 = 4$ marks

D1. Two methods take their very first step from a standing start, both with a step size of 0.03 and a first gradient of +8. The first keeps a running *total* of squared gradients; the second keeps a running *average* of them, forgetting at a rate that leaves one hundredth of the weight on the newest value. Both then divide the step by the square root of what they have accumulated; the guard constant is too small to matter.

Both start from $s_0 = 0$, so each accumulator has only g_1 in it.

$$\text{Adagrad: } s_1 = g_1^2 = 64, \quad |\Delta w_1| = \frac{0.03 \times 8}{\sqrt{64}} = \mathbf{0.03}$$

$$\text{RMSprop: } s_1 = (1 - \gamma)g_1^2 = 0.64, \quad |\Delta w_1| = \frac{0.03 \times 8}{\sqrt{0.64}} = \mathbf{0.3}$$

Adagrad's first step is **exactly** η ; RMSprop's is $\eta/\sqrt{1-\gamma} = \mathbf{10} \eta$, larger by that factor.

Why neither depends on g_1 . Put the two lines side by side. Adagrad divides ηg_1 by $\sqrt{g_1^2} = |g_1|$, and RMSprop divides it by $|g_1|\sqrt{1-\gamma}$. Either way the magnitude of g_1 cancels top and bottom, leaving only its *sign* to say which way to move. That is what makes the very first step of an adaptive method a property of the settings rather than of the data — and it is the same cancellation that gives Adam its first step of exactly η . From the second step on the gradients no longer cancel, because s then carries history.

D2. A spam filter is trained by making one quantity as small as possible, and is then described to the client by a completely different number — the fraction of messages it gets right. **(a)** Explain why these are two different quantities rather than two names for one. **(b)** State which of them you use to decide the model is finished, and why.

(a) They are different because they answer different questions. The *loss* is what the optimiser minimises, so it must be smooth and differentiable — you need a gradient to descend. The *metric* is what the decision is made on, and it need be neither: accuracy is a step function with zero gradient almost everywhere, which is precisely why it cannot be trained on. They also reward different things. Cross-entropy cares how *confident* a correct prediction was and punishes confident mistakes without bound; accuracy only asks which side of the threshold each prediction landed. A model can lower its cross-entropy without a single prediction changing side, and can improve its accuracy while its cross-entropy gets worse.

(b) The **metric**, computed on held-out data. The slogan is worth keeping: *use the loss to train, use the metric to decide*. And do not compare two models by their losses if those losses are of different kinds — pick one metric and compute it for both on the same held-out rows.

Part E — Reasoning

4 marks

E1. Someone claims, flatly, that the method which adapts each coordinate separately always beats the one that merely carries momentum. You have one afternoon and a laptop. Design the smallest experiment that would actually settle it.

There is no single right answer here, so what follows is one experiment that would work, and the reasoning that makes it work.

(i) What to run it on. The claim says *always*, so a single dataset cannot establish it and a single counter-example can destroy it. Run at least two problems of deliberately different character — one dense and well conditioned, one sparse or badly conditioned — because that is exactly the axis on which the two methods are known to differ.

(ii) What to measure, and when. Final held-out loss (or the metric you actually care about) at a fixed budget — a fixed number of epochs, or a fixed wall-clock time, fixed in advance. Repeat over a few seeds and report the spread: two runs differing by less than the seed-to-seed variation have not differed at all.

(iii) The thing that must be held equal. The learning rate must be tuned separately for each optimiser. This is the whole item. Adam and momentum have different best learning rates — they are not even on the same scale — so running both at one shared η measures only which one happens to suit that η . Sweep η for each, take each method's best, and compare those. Everything else (initialisation, batch size, budget, seeds, data splits) is held fixed.

(iv) What would decide it. Accept the claim only if Adam wins on *every* problem by more than the seed-to-seed spread — that is what *always* demands. Reject it the moment momentum wins on even one, by more than that spread. Anything in between is the honest answer that the claim is too strong: it depends on the problem.

CSAI2017P — Applied Machine Learning (Theory)

Theory Quiz 3 — Solutions • Set G

Maximum marks 20 • 15 questions on four pages

These are the answers to **Set G**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.

Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

5 × 1 = 5 marks

A1. (a) the first is larger

Thirteen cards are hearts and four are aces, so 13/52 against 4/52.

A2. (d) the first is larger

1, 5, 9, 13, 17 is five numbers; *adam* is four letters.

A3. (c) $B = 1$ makes $32\times$ as many

With $B = 1$ there are n updates per epoch; with $B = 32$ there are $n/32$. The ratio is 32, and it is why small batches make so much more progress per pass.

A4. (b) the second inflates the reported score

Fitted inside each fold, the scaler only ever sees training rows and the score is honest. Fitted once on everything, it has seen the held-back rows and the score is inflated.

A5. (b) a sum, against an average

Adagrad sums, RMSprop averages. One symbol changes; the consequence is that Adagrad's step decays forever and RMSprop's does not.

Part B — Fill in the blanks

4 × 1 = 4 marks

B1. $3w^2$

Bring the exponent down and reduce it by one: $\frac{d}{dw} w^n = n w^{n-1}$, so w^3 differentiates to $3w^2$. Note the derivative is a *function* of w — it is a different number at every point, which is exactly why gradient descent has to re-evaluate it at each step rather than computing it once.

B2. $\sqrt{B}$

A mini-batch gradient is an average of B independent single-example gradients, and averaging B independent quantities divides their standard deviation by $\sqrt{B}$. Measured on the diabetes data: the RMS error was 6.32 at $B = 1$ and 1.06 at $B = 32$, a factor of 5.9 against the predicted $\sqrt{32} = 5.66$. The cost, though, grows like B — which is why $B = 32$ is a good default and $B = 4096$ usually is not.

B3. $\sqrt{t}$

Adagrad's accumulator is a *sum*: $s_t = s_{t-1} + g_t^2$, so it can only ever grow. The step is divided by $\sqrt{s_t}$, and if the gradients stay roughly the same size then s_t grows like t and the step shrinks like $1/\sqrt{t}$ — forever, whether or not you have arrived. Measured: the effective step fell 9.4× over 2800 updates and was still falling. RMSprop's one-line fix is to make s_t an average instead, so it can come back down.

B4. η

Both of Adam's running averages start at zero, so both are biased towards zero early on. Dividing m by $1 - \beta_1^t$ and v by $1 - \beta_2^t$ removes exactly that bias. At $t = 1$ this leaves $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so the step is $\eta g_1 / \sqrt{g_1^2} = \eta \cdot \text{sign}(g_1)$: size exactly η , whatever the gradient was. Without the correction the first step is $\sqrt{1 - \beta_2}/(1 - \beta_1)$ times too big — 3.16× at the usual settings.

Part C — True or false $3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

C1. *Between a model with leakage and one without, it is the leaking model that reports the lower held-out score.*

False — and this is what makes leakage dangerous. Leakage means the model saw, at training time, information it will not have at prediction time. That makes the held-out score *better* than the truth, not worse. A disappointing score is an ordinary problem you will go and fix; a suspiciously good one is the problem you will ship. The lecture's demonstration scored 81% on pure noise.

C2. *Between an adaptive optimiser and momentum, each at its own tuned learning rate, the adaptive one always finishes lower.*

False. Measured on `load_diabetes` with each optimiser at its own tuned learning rate, all six landed within two per cent of each other — and plain momentum won. Adaptive methods shine where coordinates genuinely differ: on the sparse problem the same comparison spanned a factor of eight. What they reliably buy is not a better optimum but a *wider target*: Adagrad worked over 2.24 decades of learning rate against plain SGD's 1.31. That is worth a great deal when you have one afternoon to tune.

C3. *Between momentum and Adam on a ravine, it can be momentum that needs fewer iterations.*

True, and it follows from what each method measures. A ravine is a problem of *curvature* — the loss bends far more steeply across the valley than along it. Adaptive methods do not measure curvature; they measure gradients. Momentum, by accumulating velocity along the consistent direction and cancelling it across the oscillating one, attacks the geometry directly. Measured: 135 iterations for momentum against 262 for Adam.

Part D — Short answer $2 \times 2 = 4$ marks

D1. Compare the *first* step taken by Adagrad with the first step taken by RMSprop, both from $s_0 = 0$, both with $\eta = 0.6$, and both at a point where the gradient is $g_1 = -3$. Adagrad accumulates $s_t = s_{t-1} + g_t^2$; RMSprop accumulates $s_t = \gamma s_{t-1} + (1 - \gamma)g_t^2$ with $\gamma = 0.8$. Both apply $w \leftarrow w - \eta g_t / \sqrt{s_t}$; take ε as negligible.

Both start from $s_0 = 0$, so each accumulator has only g_1 in it.

$$\text{Adagrad: } s_1 = g_1^2 = 9, \quad |\Delta w_1| = \frac{0.6 \times 3}{\sqrt{9}} = \mathbf{0.6}$$

$$\text{RMSprop: } s_1 = (1 - \gamma)g_1^2 = 1.8, \quad |\Delta w_1| = \frac{0.6 \times 3}{\sqrt{1.8}} = \mathbf{1.341641}$$

Adagrad's first step is **exactly** η ; RMSprop's is $\eta / \sqrt{1 - \gamma} = \mathbf{2.236068} \eta$, larger by that factor.

Why neither depends on g_1 . Put the two lines side by side. Adagrad divides ηg_1 by $\sqrt{g_1^2} = |g_1|$, and RMSprop divides it by $|g_1| \sqrt{1 - \gamma}$. Either way the magnitude of g_1 cancels top and bottom, leaving only its *sign* to say which way to move. That is what makes the very first step of an adaptive method a property of the settings rather than of the data — and it is the same cancellation that gives Adam its first step of exactly η . From the second step on the gradients no longer cancel, because s then carries history.

D2. Compare the two numbers attached to one classifier: the cross-entropy its training loop minimised, and the accuracy quoted to the client. **(a)** Explain why these are two different quantities rather than two names for one. **(b)** State which of them you use to decide the model is finished, and why.

(a) They are different because they answer different questions. The *loss* is what the optimiser minimises, so it must be smooth and differentiable — you need a gradient to descend. The *metric* is what the decision is made on, and it need be neither: accuracy is a step function with zero gradient almost everywhere, which is precisely why it cannot be trained on. They also reward different things. Cross-entropy cares how *confident* a correct prediction was and punishes confident mistakes without bound; accuracy only asks which side of the threshold each prediction landed. A model can lower its cross-entropy without a single prediction changing side, and can improve its accuracy while its cross-entropy gets worse.

(b) The **metric**, computed on held-out data. The slogan is worth keeping: *use the loss to train, use the metric to decide*. And do not compare two models by their losses if those losses are of different kinds — pick one metric and compute it for both on the same held-out rows.

Part E — Reasoning

4 marks

E1. Two colleagues disagree. One says “*Adam always beats plain gradient descent with momentum*”; the other says it depends. You have one afternoon and a laptop. Design the smallest experiment that would actually settle it between them.

There is no single right answer here, so what follows is one experiment that would work, and the reasoning that makes it work.

(i) What to run it on. The claim says *always*, so a single dataset cannot establish it and a single counter-example can destroy it. Run at least two problems of deliberately different character — one dense and well conditioned, one sparse or badly conditioned — because that is exactly the axis on which the two methods are known to differ.

(ii) What to measure, and when. Final held-out loss (or the metric you actually care about) at a fixed budget — a fixed number of epochs, or a fixed wall-clock time, fixed in advance. Repeat over a few seeds and report the spread: two runs differing by less than the seed-to-seed variation have not differed at all.

(iii) The thing that must be held equal. The learning rate must be tuned separately for each optimiser. This is the whole item. Adam and momentum have different best learning rates — they are not even on the same scale — so running both at one shared η measures only which one happens to suit that η . Sweep η for each, take each method’s best, and compare those. Everything else (initialisation, batch size, budget, seeds, data splits) is held fixed.

(iv) What would decide it. Accept the claim only if Adam wins on *every* problem by more than the seed-to-seed spread — that is what *always* demands. Reject it the moment momentum wins on even one, by more than that spread. Anything in between is the honest answer that the claim is too strong: it depends on the problem.

CSAI2017P — Applied Machine Learning (Theory)

Theory Quiz 3 — Solutions • Set H

Maximum marks **20** • 15 questions on four pages

These are the answers to **Set H**, with the reasoning behind each one. Find the set letter on your own paper and work from the matching four pages. Sets were handed out at random and every set covers the same material, so if you want the extra practice the other sets are worth reading too.

Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. If you reached a different answer and can show your reasoning, bring your script and let us go through it.

Part A — Multiple choice

$5 \times 1 = 5$ marks

A1. (b) $1/4$

The worked line gives the rule: count the favourable cases over the total.

A2. (a) 5

Same rule as the worked line, with a different step: 1, 5, 9, 13, 17 stops before 20.

A3. (d) 125

n/B in both cases — $100/10 = 10$ in the worked line, $4000/32 = 125$ here.

A4. (c) too good

The contrast is the point: the same scaler gives an honest score inside the fold and a flattering one outside it.

A5. (c) average of squared gradients

A sum against an average. Adagrad's total only grows; RMSprop's weights sum to one, so its accumulator stays on the scale of a typical g^2 and can fall again.

Part B — Fill in the blanks

$4 \times 1 = 4$ marks

B1. $3w^2$

Bring the exponent down and reduce it by one: $\frac{d}{dw} w^n = n w^{n-1}$, so w^3 differentiates to $3w^2$. Note the derivative is a *function* of w — it is a different number at every point, which is exactly why gradient descent has to re-evaluate it at each step rather than computing it once.

B2. $\sqrt{B}$

A mini-batch gradient is an average of B independent single-example gradients, and averaging B independent quantities divides their standard deviation by $\sqrt{B}$. Measured on the diabetes data: the RMS error was 6.32 at $B = 1$ and 1.06 at $B = 32$, a factor of 5.9 against the predicted $\sqrt{32} = 5.66$. The cost, though, grows like B — which is why $B = 32$ is a good default and $B = 4096$ usually is not.

B3. $\sqrt{t}$

Adagrad's accumulator is a *sum*: $s_t = s_{t-1} + g_t^2$, so it can only ever grow. The step is divided by $\sqrt{s_t}$, and if the gradients stay roughly the same size then s_t grows like t and the step shrinks like $1/\sqrt{t}$ — forever, whether or not you have arrived. Measured: the effective step fell $9.4\times$ over 2800 updates and was still falling. RMSprop's one-line fix is to make s_t an average instead, so it can come back down.

B4. η

Both of Adam's running averages start at zero, so both are biased towards zero early on. Dividing m by $1 - \beta_1^t$ and v by $1 - \beta_2^t$ removes exactly that bias. At $t = 1$ this leaves $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so the step is $\eta g_1 / \sqrt{g_1^2} = \eta \cdot \text{sign}(g_1)$: size exactly η , whatever the gradient was. Without the correction the first step is $\sqrt{1 - \beta_2}/(1 - \beta_1)$ times too big — $3.16\times$ at the usual settings.

Part C — True or false $3 \times 1 = 3$ marks

The statement is reproduced first, then the answer and why. A reason was optional on the paper, but writing one is how you find out whether you actually had the idea.

- C1.** An honest pipeline gives a believable held-out score. *A leaking one therefore gives a score that is believably low.*
False — and this is what makes leakage dangerous. Leakage means the model saw, at training time, information it will not have at prediction time. That makes the held-out score *better* than the truth, not worse. A disappointing score is an ordinary problem you will go and fix; a suspiciously good one is the problem you will ship. The lecture’s demonstration scored 81% on pure noise.
- C2.** On a sparse problem, Adagrad beat plain SGD by a factor of eight. *Therefore an adaptive optimiser, tuned, beats momentum on any problem.*
False. Measured on `load_diabetes` with each optimiser at its own tuned learning rate, all six landed within two per cent of each other — and plain momentum won. Adaptive methods shine where coordinates genuinely differ: on the sparse problem the same comparison spanned a factor of eight. What they reliably buy is not a better optimum but a *wider target*: Adagrad worked over 2.24 decades of learning rate against plain SGD’s 1.31. That is worth a great deal when you have one afternoon to tune.
- C3.** Adaptive methods measure gradients, and a ravine is a problem of curvature. *So on a ravine, momentum can beat Adam.*
True, and it follows from what each method measures. A ravine is a problem of *curvature* — the loss bends far more steeply across the valley than along it. Adaptive methods do not measure curvature; they measure gradients. Momentum, by accumulating velocity along the consistent direction and cancelling it across the oscillating one, attacks the geometry directly. Measured: 135 iterations for momentum against 262 for Adam.

Part D — Short answer $2 \times 2 = 4$ marks

D1. From $s_0 = 0$ with $\eta = 1$ and $g_1 = -2$, Adagrad has $s_1 = 4$, so its first step is $1 \times (-2)/2 = -1$: size 1, which is η . Now take $\eta = 0.05$ and $g_1 = +12$, with RMSprop's $\gamma = 0.95$. Adagrad accumulates $s_t = s_{t-1} + g_t^2$; RMSprop accumulates $s_t = \gamma s_{t-1} + (1 - \gamma)g_t^2$. Both apply $w \leftarrow w - \eta g_t / \sqrt{s_t}$; take ε as negligible.

Both start from $s_0 = 0$, so each accumulator has only g_1 in it.

$$\text{Adagrad: } s_1 = g_1^2 = 144, \quad |\Delta w_1| = \frac{0.05 \times 12}{\sqrt{144}} = \mathbf{0.05}$$

$$\text{RMSprop: } s_1 = (1 - \gamma)g_1^2 = 7.2, \quad |\Delta w_1| = \frac{0.05 \times 12}{\sqrt{7.2}} = \mathbf{0.223607}$$

Adagrad's first step is **exactly** η ; RMSprop's is $\eta / \sqrt{1 - \gamma} = \mathbf{4.472136} \eta$, larger by that factor.

Why neither depends on g_1 . Put the two lines side by side. Adagrad divides ηg_1 by $\sqrt{g_1^2} = |g_1|$, and RMSprop divides it by $|g_1| \sqrt{1 - \gamma}$. Either way the magnitude of g_1 cancels top and bottom, leaving only its *sign* to say which way to move. That is what makes the very first step of an adaptive method a property of the settings rather than of the data — and it is the same cancellation that gives Adam its first step of exactly η . From the second step on the gradients no longer cancel, because s then carries history.

D2. A ruler and a thermometer both produce numbers, but they are not two names for one measurement. A classifier's cross-entropy and its accuracy are likewise two different things. **(a)** Explain why. **(b)** State which of them you use to decide the model is finished, and why.

(a) They are different because they answer different questions. The *loss* is what the optimiser minimises, so it must be smooth and differentiable — you need a gradient to descend. The *metric* is what the decision is made on, and it need be neither: accuracy is a step function with zero gradient almost everywhere, which is precisely why it cannot be trained on. They also reward different things. Cross-entropy cares how *confident* a correct prediction was and punishes confident mistakes without bound; accuracy only asks which side of the threshold each prediction landed. A model can lower its cross-entropy without a single prediction changing side, and can improve its accuracy while its cross-entropy gets worse.

(b) The **metric**, computed on held-out data. The slogan is worth keeping: *use the loss to train, use the metric to decide*. And do not compare two models by their losses if those losses are of different kinds — pick one metric and compute it for both on the same held-out rows.

Part E — Reasoning

4 marks

E1. Worked analogue. To settle “drug A cures faster than drug B” you would not give A to the fit patients and B to the frail ones — you would hold everything but the drug equal, and decide in advance what difference would count.

Now settle this one. A classmate asserts: “Adam always beats plain gradient descent with momentum.” You have one afternoon and a laptop. Design the smallest experiment that would actually settle it.

There is no single right answer here, so what follows is one experiment that would work, and the reasoning that makes it work.

(i) What to run it on. The claim says *always*, so a single dataset cannot establish it and a single counter-example can destroy it. Run at least two problems of deliberately different character — one dense and well conditioned, one sparse or badly conditioned — because that is exactly the axis on which the two methods are known to differ.

(ii) What to measure, and when. Final held-out loss (or the metric you actually care about) at a fixed budget — a fixed number of epochs, or a fixed wall-clock time, fixed in advance. Repeat over a few seeds and report the spread: two runs differing by less than the seed-to-seed variation have not differed at all.

(iii) The thing that must be held equal. The learning rate must be tuned separately for each optimiser. This is the whole item. Adam and momentum have different best learning rates — they are not even on the same scale — so running both at one shared η measures only which one happens to suit that η . Sweep η for each, take each method’s best, and compare those. Everything else (initialisation, batch size, budget, seeds, data splits) is held fixed.

(iv) What would decide it. Accept the claim only if Adam wins on *every* problem by more than the seed-to-seed spread — that is what *always* demands. Reject it the moment momentum wins on even one, by more than that spread. Anything in between is the honest answer that the claim is too strong: it depends on the problem.